A10 Deployment

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track A: Databricks pipeline engineering 2
A10 Deployment 3
A10-L1 Git folders, one branch per change, and no secrets in the repository 3
A10-L2 Declarative Automation Bundles and parameterised jobs 7
A10-L3 CI: lint, type-check, test and validate on every pull request 12
Track A best-practice checklist 17

Appendix: solutions and answer keys 18

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A10 Deployment

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Git folders, Databricks Asset Bundles, parameterised jobs and CI.

A10-L1 Git folders, one branch per change, and no secrets in the repository

Concept

Production pipeline code lives in Git, not in someone's workspace folder. Databricks **Git folders** connect a workspace folder to a remote repository, so you can edit, commit, pull and switch branches from the workspace.

The workflow Databricks documents:

- Each developer clones the repository into their own Git folder under `/Workspace/Users/<email>/` and works on their own branch.
- Use a **trunk-based** strategy: short-lived feature branches off `main`, merged through a pull request with review and CI checks.
- **Production Git folders** are different. An admin creates them outside user folders. They hold a deployment branch, and only automation updates them, after a pull request merges. Nobody edits code there by hand.
- For deploying jobs and their configuration together, Databricks recommends Declarative Automation Bundles (lesson A10-L2).

Store notebooks as source files (`.py` with `# Databricks notebook source` headers, or `.ipynb`). Put reusable functions in plain Python modules that notebooks import, so tests can import them too.

Secrets never go into the repository. Not in code, not in config files, not in notebook outputs. Git keeps every version forever, and removing a token in a later commit does not remove it from history. Rotate any secret that was ever committed.

In Databricks, read secrets at run time:

```
api_key = dbutils.secrets.get(scope="research", key="supplier_api_key")
```

Databricks redacts secret values read this way, showing `[REDACTED]` if you print them. Redaction does not stop deliberate transformations, so never try to display a secret. In CI, secrets come from the CI system's secret store as environment variables.

Add a `.gitignore` for local files (`.venv/`, `__pycache__/`, `.env`), so credentials and environments never get staged by accident.

Why it matters

A developer tests a supplier API from a notebook with a hardcoded token, then commits the notebook to share the code. A week later they replace the token with a secret lookup in a new commit, believing the problem is fixed. The repository is later mirrored to a partner, and the token is still in the history. The supplier revokes the key after detecting unusual use, and every ingestion job that depends on it fails for two days.

With `dbutils.secrets.get` from the first line, there is nothing to leak.

Worked example: weak code

```
1 # silver_nat_transform (edited directly in the production folder)
2 SUPPLIER_TOKEN = "example-token-not-real-1234"
3 CATALOG = "research_prod"
4
5 def fetch_deliveries():
6     return requests.get(URL, headers={"Authorization": f"Bearer
    {SUPPLIER_TOKEN}"})
```

1. **Line 1.** Hand edits in production skip review and CI, and drift from Git.
2. **Line 2.** A token in code. Once committed, it stays in Git history for good.
3. **Line 3.** Environment hardcoded, so the same code cannot run in dev.

Worked example: strong code

```
1 # Databricks notebook source
1 # silver_nat_transform: edited on a feature branch, deployed by CI after
    review.
3 from pipelines.config import load_run_config
2 from pipelines.supplier import fetch_deliveries
5
3 cfg = load_run_config(dbutils.widgets.get("config_path"))
4 token = dbutils.secrets.get(scope="research", key="supplier_api_key")
8 deliveries = fetch_deliveries(cfg.supplier_url, token)
```

1. **Line 2.** Changes reach production only through a reviewed, tested merge.
2. **Line 4.** Logic lives in importable modules that unit tests can reach.
3. **Line 6.** The environment comes from a job parameter, not from the code.
4. **Line 7.** Read at run time from a secret scope. Nothing sensitive in the repository.

Code flows from a branch, through review and CI, to production. Secrets flow from a secret store at run time, never through Git.

Recap card

- **Where do developers work in Databricks Git folders?** Their own clone under /Workspace/Users/<email>/, on their own branch.
- **Who updates production Git folders?** Only automation, after a pull request merges into the deployment branch.
- **Read a secret in a notebook** dbutils.secrets.get(scope="...", key="..."). Output shows [REDACTED].
- **A token was committed, then removed. Is it safe?** No. Git history keeps it. Rotate the token.

Practice

Exercise 1. Multiple choice (level 1)

Who should update the production Git folder?

- a) Any developer, after testing their change
- b) Automation, after a pull request merges into the deployment branch
- c) The team lead, by editing notebooks there directly
- d) Nobody: production reads from developers' user folders

Why?

- r1) Production changes must pass review and CI, and automation makes that the only path.
- r2) Git folders cannot be edited by people.

Exercise 2. Fill the gap (level 2)

Read the supplier API key from the `research` secret scope at run time.

```
token = dbutils._____(1).get(scope="research", key="_____(2)")
```

Exercise 3. Spot the bug (level 3)

A reviewer blocks this pull request. Tap the line that must never be committed, then rewrite the module.

What you need to know.

`requests.get(url, headers={...}, timeout=30)` sends an HTTP GET request and returns a `requests.Response`, which holds the status code and the body. An `Authorization: Bearer <token>` header sends the caller's access token. `timeout` stops the call from waiting forever.

```
import requests

SUPPLIER_URL = "https://supplier.example.org/api/deliveries"
```

```
SUPPLIER_TOKEN = "example-token-not-real-5678"


def fetch_deliveries() -> requests.Response:
    """Fetch the delivery list from the supplier API."""
    return requests.get(SUPPLIER_URL, headers={"Authorization": f"Bearer {SUPPLIER_TOKEN}"}, timeout=30)
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Put in order (level 4)

Order the steps for one change to the silver pipeline.

- Push and open a pull request
- Create a feature branch from main in your Git folder
- CI runs lint, type checks and tests
- Merge to main; automation deploys to production
- A colleague reviews and approves
- Change the code and add or update tests

Exercise 5. Architecture design (level 5)

Decide where each item lives.

Options: Secret scope, read at run time, Git repository, Job parameter.

1. The supplier API token
2. The functions that clean and merge silver
3. Which catalog a run writes to (research_dev or research_prod)
4. Your local .venv folder

Sources

- Git folders: per-user branches under /Workspace/Users/, trunk-based strategy, production Git folders updated only by automation, bundles recommended for deployment. (verified: https://docs.databricks.com/aws/en/repos/ci-cd (checked 2026-09-25))
- dbutils.secrets.get(scope, key) reads a secret; values are shown as [REDACTED]; redaction does not prevent deliberate transformations. (verified: https://docs.databricks.com/aws/en/security/secrets/ (checked 2026-09-25))

A10-L2 Declarative Automation Bundles and parameterised jobs

Concept

A job built by clicking in the UI exists only in one workspace, with no history and no review. **Declarative Automation Bundles** describe jobs, their tasks, parameters and notifications as YAML files in your repository. Databricks renamed them from Databricks Asset Bundles on 16 March 2026. The `databricks bundle` commands and `databricks.yml` did not change.

A bundle has one `databricks.yml` at its root:

```yaml
bundle:
  name: research_pipelines

variables:
  catalog:
    description: Catalog the pipelines write to.

resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      parameters:
        - name: catalog
          default: ${var.catalog}
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: ./notebooks/silver_nat_transform.py

targets:
  dev:
    mode: development
    default: true
    variables:
      catalog: research_dev
  prod:
    mode: production
    variables:
      catalog: research_prod
```

- **Targets** are environments. One code base deploys to each with different values.
- **Variables** hold what differs per target. They resolve from `--var`, then `BUNDLE_VAR_` environment variables, then target mappings, then defaults.

- **Job parameters** reach notebook tasks, which read them with `dbutils.widgets.get("catalog")`. Dynamic references such as `{{job.run_id}}` fill in values at run time.
- `mode: development` prefixes job names with `[dev <your name>]` and pauses schedules, so your test deployment never runs on the production timetable.

The commands:

```
databricks bundle validate -t dev
databricks bundle deploy -t dev
databricks bundle run -t dev nat_silver_pipeline
```

Deploy to `prod` only from CI, as a service principal (lesson A10-L3).

Why it matters

A team manages its jobs in the UI. Someone changes the production silver job's notebook path to test a branch, meaning to change it back. They forget. For a week, production runs a half-finished branch. Nobody can say what changed or when, because the UI keeps no diff and nothing was reviewed.

With the job in a bundle, the path is a line in `databricks.yml`. Changing it needs a pull request, CI validates it, and Git shows exactly what changed.

Worked example: weak code

```
resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: /Workspace/Users/sam.jones@example.org/
    silver_nat_transform
            base_parameters:
              catalog: research_prod
```

1. **Line 1.** No targets: there is no way to deploy the same job to dev.
2. **Line 8.** Points at one person's folder: unreviewed code, and it breaks when they leave.
3. **Line 10.** Production hardcoded: every deployment of this bundle writes to prod.

Worked example: strong code

```
bundle:
  name: research_pipelines

variables:
```

```yaml
  catalog:
    description: Catalog the pipelines write to.

resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      parameters:
        - name: catalog
          default: ${var.catalog}
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: ./notebooks/silver_nat_transform.py
      email_notifications:
        on_failure:
          - data-oncall@example.org

targets:
  dev:
    mode: development
    default: true
    variables:
      catalog: research_dev
  prod:
    mode: production
    variables:
      catalog: research_prod
```

1. **Line 14.** The job parameter takes its value from the target's variable.
2. **Line 18.** A path inside the bundle, deployed from the reviewed commit.
3. **Line 25.** Dev deployments get a [dev name] prefix and paused schedules.
4. **Line 32.** Production values live only in the prod target.

One reviewed file describes the job for every environment. The target decides where it writes.

Recap card

- **What are Declarative Automation Bundles?** Jobs and their settings as YAML in Git (formerly Databricks Asset Bundles). Same CLI: databricks bundle.
- **Targets and variables** Targets are environments. Variables hold what differs, such as the catalog, set per target.
- **What does mode: development do?** Prefixes resources with [dev <user>] and pauses schedules and triggers.
- **Read a job parameter in a notebook** dbutils.widgets.get("name"). Validate it before using it.

Practice

Exercise 1. Multiple choice (level 1)

What changed when Databricks Asset Bundles became Declarative Automation Bundles on 16 March 2026?

- a) The CLI command changed from `databricks bundle` to `databricks dab`.
- b) The name. The `databricks bundle` CLI and existing configuration keep working unchanged.
- c) `databricks.yml` was renamed `automation.yml`.
- d) Targets were removed in favour of workspaces.

Why?

- r1) Databricks describes the rename as non-breaking.
- r2) Bundles were replaced by a new product.

Exercise 2. Fill the gap (level 2)

Complete the targets so dev deployments are prefixed and paused, and each target sets its own catalog.

```yaml
targets:
  dev:
    mode: _____(1)
    default: true
    variables:
      catalog: research_dev
  prod:
    mode: _____(2)
    variables:
      _____(3): research_prod
```

Exercise 3. Spot the bug (level 3)

A developer runs `databricks bundle deploy -t dev`, and the next morning production data has changed. Tap the line, then fix the job so the catalog comes from the target.

```yaml
resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      parameters:
        - name: catalog
          default: research_prod
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: ./notebooks/silver_nat_transform.py
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Write the function (level 4)

The notebook for this job must read its `catalog` and `wave_id` job parameters. Write `read_job_parameters(widgets)`: `widgets` behaves like `dbutils.widgets` (it has `.get(name)`). Return a dict with both values. Raise `ValueError` when `catalog` is not `research_dev` or `research_prod`, or when `wave_id` is empty.

Your code must pass these checks:

- Return both parameters as read from widgets.get.
- An unexpected catalog must raise ValueError before any data moves.
- An empty wave_id must raise ValueError.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Module constant.
ALLOWED_CATALOGS = ...

# Read and validate the job parameters passed to this notebook.
def read_job_parameters(widgets: object) -> dict[str, str]: ...
```

Starter code:

```
def read_job_parameters(widgets: object) -> dict[str, str]:
    ...
```

Exercise 5. Put in order (level 5)

Order the commands to try a bundle change in dev.

- databricks bundle deploy -t dev
- databricks bundle run -t dev nat_silver_pipeline
- Check the run log and QA results for the dev run
- databricks bundle validate -t dev

Sources

- Databricks Asset Bundles were renamed Declarative Automation Bundles on 16 March 2026; the change is non-breaking. (verified: https://docs.databricks.com/aws/en/release-notes/dev-tools/bundles (checked 2026-09-25))
- databricks.yml structure: bundle, variables, resources.jobs with parameters and tasks, targets with mode; variable resolution order. (search-only: https://docs.databricks.com/aws/en/dev-tools/bundles/

settings and https://docs.databricks.com/aws/en/dev-tools/bundles/variables)

- Development mode prefixes resources with [dev ${workspace.current_user.short_name}] and pauses schedules and triggers. (verified: https://docs.databricks.com/aws/en/dev-tools/bundles/deployment-modes (checked 2026-09-25))
- Notebook tasks read job parameters with dbutils.widgets.get. (verified: https://docs.databricks.com/aws/en/jobs/parameter-use (checked 2026-09-25))

A10-L3 CI: lint, type-check, test and validate on every pull request

Concept

Continuous integration (CI) runs the same checks on every change, before a human reviews it. For a Databricks pipeline repository, run four checks on each pull request:

1. **Lint and format**: `ruff check .` and `ruff format --check .`
2. **Type-check**: `mypy`
3. **Unit tests**: `pytest`, with a local SparkSession (lesson A9-L2). Install Java in the CI runner first.
4. **Bundle validation**: `databricks bundle validate -t dev`. It catches YAML errors and broken references before anything is deployed.

On merge to `main`, a separate job deploys: `databricks bundle deploy -t prod`. Only CI deploys to production, and it does so as a **service principal**, never as a person.

Authenticating CI. Databricks recommends OAuth machine-to-machine authentication for a service principal. The CLI reads the workspace and credentials from environment variables, filled from the CI system's secret store:

```
env:
  DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
  DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
  DATABRICKS_CLIENT_SECRET: ${{ secrets.DATABRICKS_CLIENT_SECRET }}
```

Workload identity federation goes a step further: with `DATABRICKS_AUTH_TYPE: github-oidc`, GitHub proves the workflow's identity, and no client secret is stored at all. Prefer it where your workspace supports it.

Two rules keep CI trustworthy:

- **CI must be able to fail.** A test step with `continue-on-error: true`, or a `|| true` after a command, reports success whatever happens.
- **Protect `main`.** Require the CI checks to pass before a pull request can merge. Otherwise CI only reports problems, and nothing stops them.

Install the CLI with the `databricks/setup-cli` action, and pin its version so a CLI release cannot change your pipeline's behaviour overnight.

Why it matters

A team adds CI but marks the test step `continue-on-error: true` "until the flaky tests are fixed". Six weeks later, every pull request shows a green tick. A change that breaks the silver MERGE key merges on a Friday. The failing test had run and failed, but the step's failure no longer failed the job. Production double-counts a wave over the weekend.

With the test step allowed to fail and `main` protected by required checks, the pull request could not have merged.

Worked example: weak code

```yaml
on: push
jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest
        continue-on-error: true
      - run: databricks bundle deploy -t prod
        env:
          DATABRICKS_TOKEN: dapi-REDACTED-EXAMPLE
```

1. **Line 9.** Failing tests no longer fail the job.
2. **Line 10.** Deploys every push, on every branch, straight to production.
3. **Line 12.** A credential written into the workflow file: committed to Git.

Worked example: strong code

```yaml
name: CI
on:
  pull_request:
  push:
    branches: [main]

jobs:
  checks:
```

```yaml
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
      - run: pip install -e . --group test
      - run: ruff check . && ruff format --check .
      - run: mypy
      - run: pytest
      - uses: databricks/setup-cli@main
        with:
          version: 0.270.0
      - run: databricks bundle validate -t dev
        env:
          DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
          DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
          DATABRICKS_CLIENT_SECRET: $
{{ secrets.DATABRICKS_CLIENT_SECRET }}

deploy:
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    needs: checks
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: databricks/setup-cli@main
        with:
          version: 0.270.0
      - run: databricks bundle deploy -t prod
        env:
          DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
          DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
          DATABRICKS_CLIENT_SECRET: $
{{ secrets.DATABRICKS_CLIENT_SECRET }}
```

1. **Line 20.** Every check can fail the job. Nothing is marked continue-on-error.
2. **Line 26.** Validates the bundle on every pull request, before review.
3. **Line 33.** Deploys only after a merge to main.
4. **Line 34.** Deployment waits for every check to pass.
5. **Line 41.** Deploys as the service principal, with credentials from the CI secret store.

Every pull request is linted, typed, tested and validated. Only a merge to main deploys, as a service principal, with nothing secret in the file. The CLI version shown is an example: pin the version you tested.

Recap card

- **Four CI checks for a pipeline repository** ruff (lint and format), mypy, pytest with local Spark, databricks bundle validate.
- **Who deploys to production?** CI, after a merge to main, as a service principal.
- **OAuth M2M environment variables** DATABRICKS_HOST, DATABRICKS_CLIENT_ID, DATABRICKS_CLIENT_SECRET from the CI secret store.
- **Two ways CI lies** continue-on-error: true, and command || true. Both turn failures green.

Practice

Exercise 1. Multiple choice (level 1)

Which identity should CI use to run `databricks bundle deploy -t prod`?

- a) The personal access token of the most senior engineer
- b) A service principal, authenticated with OAuth M2M or workload identity federation
- c) Whoever opened the pull request
- d) An anonymous connection

Why?

- r1) A service principal is not tied to a person, and its access can be scoped to exactly what deployment needs.
- r2) Personal tokens cannot deploy bundles.

Exercise 2. Fill the gap (level 2)

Complete the environment for the bundle deploy step, using OAuth M2M with values from the CI secret store.

```
env:
  DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
  DATABRICKS_____(1): ${{ secrets.DATABRICKS_CLIENT_ID }}
  DATABRICKS_____(2): ${{ secrets.DATABRICKS_CLIENT_SECRET }}
```

Exercise 3. Spot the bug (level 3)

Every pull request shows a green tick, including ones that broke the MERGE. Tap the line, then fix the step.

```
steps:
  - uses: actions/checkout@v4
  - run: pip install -e . --group dev
```

```
    - run: ruff check .
    - run: pytest
      continue-on-error: true
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Put in order (level 4)

Order the checks in the pull request job, cheapest and fastest first.

- ruff check . and ruff format --check .
- mypy
- databricks bundle validate -t dev
- pytest with a local SparkSession

Exercise 5. Architecture design (level 5)

Configure the repository and workflow. Choose each setting.

Options: On every pull request, and on push to main, Nightly, When someone remembers.

1. When do lint, types, tests and validation run?
2. When does the prod deployment run?
3. Branch protection on main
4. CI authentication

Sources

- Databricks recommends OAuth M2M for service principals in CI; environment variables DATABRICKS_HOST, DATABRICKS_CLIENT_ID and DATABRICKS_CLIENT_SECRET; github-oidc workload identity federation needs no client secret. (verified: https://docs.databricks.com/aws/en/dev-tools/auth/oauth-m2m and https://docs.databricks.com/aws/en/dev-tools/bundles/authentication (checked 2026-09-25))
- databricks/setup-cli installs the Databricks CLI in GitHub Actions and accepts a version input. (search-only: https://github.com/databricks/setup-cli)
- CLI version 0.270.0 in the example is illustrative, not a verified current release. (unverified: Pin the version you have tested; check https://github.com/databricks/cli/releases.)

Track A best-practice checklist

☐ Every function has type hints, a docstring and one job.

☐ No SparkSession is created at import time. Functions receive spark as a parameter.

☐ Functions are imported as F: `from pyspark.sql import functions as F`.

☐ Table names, column maps and thresholds come from config or registry tables.

☐ Every table states its grain in a comment or table property.

☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.

☐ Every run writes one record to the run log table with an explicit schema.

☐ CRITICAL checks raise. WARN checks log and continue.

☐ Every division has a guard for zero and null.

☐ Counts come from one agg pass, not repeated count() calls.

☐ Incremental loads compare source and target on the same keys.

☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.

☐ Tests run on a local SparkSession in CI before every merge.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A10-L1 Git folders, one branch per change, and no secrets in the repository

A10-L1 exercise 1

Hints. (1) Which path includes review and tests? (2) What should be the only way into production? (3) A merge, then automation.

Answer. b) Automation, after a pull request merges into the deployment branch Because: r1) Production changes must pass review and CI, and automation makes that the only path.

Production Git folders hold the deployment branch and are updated only by automation after a merge, so every production change was reviewed and tested.

Common wrong answers.

- `a`: A developer's own testing skips review and CI. Production should only change through a merge.
- `c`: Direct edits in production drift from Git and bypass every check.
- `d`: User folders hold work in progress on personal branches. Production needs a stable deployment branch.
- `b|r2`: Right answer, wrong reason. People can edit Git folders. The rule is that production ones change only through automation.

A10-L1 exercise 2

Hints. (1) The dbutils module named after what it stores. (2) The key names the secret inside the scope. (3) `secrets` and `supplier_api_key`.

Answer.

```
token = dbutils.secrets.get(scope="research", key="supplier_api_key")
```

`dbutils.secrets.get(scope=..., key=...)` reads the secret at run time, and Databricks redacts it in notebook output.

Common wrong answers.

- g1 = `widgets`: Widgets hold job parameters, which appear in the job UI and run history. Secrets come from `dbutils.secrets`.

- g1 = `fs`: `dbutils.fs` reads files. Reading a key from a file puts it back on storage in plain text.

A10-L1 exercise 3

Hints. (1) Which line holds a credential? (2) Where should the credential come from instead? (3) Delete it and take the token as a parameter.

Answer. The bug is on line 4.

Fixed version:

```
1 import requests
2
3 SUPPLIER_URL = "https://supplier.example.org/api/deliveries"
4
5
6 def fetch_deliveries(token: str) -> requests.Response:
7     """Fetch the delivery list. token comes from dbutils.secrets.get at
    run time."""
8     return requests.get(SUPPLIER_URL, headers={"Authorization": f"Bearer
    {token}"}, timeout=30)
```

Line 4 commits a credential. Make the function take `token: str` and have the notebook read it with `dbutils.secrets.get`. Rotate the leaked token, because Git history keeps it.

Common wrong answers.

- Line 3: A public API URL is not a secret. It could even move to config, but it is not what blocks the review.
- Line 9: Line 9 uses the token. The leak is where the token is written.

A10-L1 exercise 4

Hints. (1) Nothing reaches main without review. (2) CI runs on the pull request. (3) Deployment is the last step, done by automation.

Answer.

1. Create a feature branch from main in your Git folder
2. Change the code and add or update tests
3. Push and open a pull request
4. CI runs lint, type checks and tests
5. A colleague reviews and approves
6. Merge to main; automation deploys to production

Branch, change, pull request, CI, review, merge and automatic deployment. Every production change passes the same gates.

Common wrong answers.

- Deploying before review and CI means production runs unchecked code.
- Create the branch first, so your changes never touch main directly.

A10-L1 exercise 5

Hints. (1) What must stay out of logs and history? (2) What changes per environment? (3) What is specific to one machine?

Answer.

- Credentials: a secret scope, read with dbutils.secrets.get.
- Code: Git, reviewed and tested.
- Environment choice: a job parameter or target setting, so the same code deploys to dev and prod.
- Local environments: ignored by Git. Pin dependencies in pyproject.toml instead.

Secrets in a secret scope, code in Git, environment choice as a parameter, local environments ignored.

Common wrong answers.

- token = `param`: Job parameters show in the job definition and run history. Credentials belong in a secret scope.
- token = `repo`: Git history keeps every committed secret for good.
- catalog = `repo`: Hardcoding the catalog in code means dev runs write to prod. Pass it in per environment.
- env = `commit`: A virtual environment is machine-specific and large. Commit the pinned dependency list instead.

A10-L2 Declarative Automation Bundles and parameterised jobs

A10-L2 exercise 1

Hints. (1) Was the rename breaking? (2) Did you need to change your files? (3) Only the name changed.

Answer. b) The name. The `databricks bundle` CLI and existing configuration keep working unchanged. Because: r1) Databricks describes the rename as non-breaking.

The rename is non-breaking. The CLI, `databricks.yml` and all existing configuration work as before.

Common wrong answers.

- `a`: The CLI command group is still `databricks bundle`.
- `c`: The configuration file is still `databricks.yml`.
- `d`: Targets are unchanged and remain the way to describe environments.
- `b|r2`: Right answer, wrong reason. It is the same product under a new name.

A10-L2 exercise 2

Hints. (1) The mode values are full words. (2) The prod target sets the same variable as dev. (3) `development`, `production` and `catalog`.

Answer.

```yaml
targets:
  dev:
    mode: development
    default: true
    variables:
      catalog: research_dev
  prod:
    mode: production
    variables:
      catalog: research_prod
```

`mode: development` prefixes names and pauses schedules. `mode: production` adds production checks. Each target sets the same variable to its own value.

Common wrong answers.

- g1 = `dev`: The mode values are `development` and `production`. `dev` is only the target's name.
- g3 = `catalog_name`: The variable is declared as `catalog`. A different name here sets nothing, and prod falls back to the default.

A10-L2 exercise 3

Hints. (1) Which value is the same in every target? (2) Where should environment-specific values come from? (3) Use `${var.catalog}`.

Answer. The bug is on line 7.

Fixed version:

```yaml
resources:
  jobs:
    nat_silver_pipeline:
      name: nat_silver_pipeline
      parameters:
        - name: catalog
          default: ${var.catalog}
```

```
      tasks:
        - task_key: silver
          notebook_task:
            notebook_path: ./notebooks/silver_nat_transform.py
```

A literal default of `research_prod` applies to every target, dev included. `${var.catalog}` takes the value each target sets.

Common wrong answers.

- Line 11: A relative notebook path inside the bundle is correct.
- Line 6: Declaring a catalog parameter is right. Its default value is the problem.

A10-L2 exercise 4

Hints. (1) Call `widgets.get("catalog")` and `widgets.get("wave_id")`. (2) Check the catalog against an allowed set. (3) Raise ValueError for a bad catalog or an empty wave_id. Never fill in a default.

Answer.

```python
ALLOWED_CATALOGS = {"research_dev", "research_prod"}


def read_job_parameters(widgets: object) -> dict[str, str]:
    """Read and validate the job parameters passed to this notebook.

    Args:
        widgets: dbutils.widgets, or any object with get(name).

    Returns:
        catalog and wave_id.

    Raises:
        ValueError: If the catalog is not allowed or wave_id is empty.
    """
    get = getattr(widgets, "get")
    catalog = str(get("catalog"))
    wave_id = str(get("wave_id")).strip()
    if catalog not in ALLOWED_CATALOGS:
        raise ValueError(f"catalog {catalog!r} is not one of
{sorted(ALLOWED_CATALOGS)}")
    if not wave_id:
        raise ValueError("wave_id is empty")
    return {"catalog": catalog, "wave_id": wave_id}
```

Parameters arrive as strings from outside the code. Validating them at the start turns a mistyped catalog into a clear failure, instead of a run against the wrong environment.

A10-L2 exercise 5

Hints. (1) Catch configuration errors first. (2) You can only run what is deployed. (3) Then look at the results.

Answer.

1. databricks bundle validate -t dev
2. databricks bundle deploy -t dev
3. databricks bundle run -t dev nat_silver_pipeline
4. Check the run log and QA results for the dev run

Validate, deploy, run, then check the evidence. The same sequence runs in CI for prod, without the manual run.

Common wrong answers.

- Validate first. It catches YAML and reference errors before anything is deployed.
- run starts the deployed job. Deploy the change first, or you run the old version.

A10-L3 CI: lint, type-check, test and validate on every pull request

A10-L3 exercise 1

Hints. (1) Who should own production deployments? (2) What identity is not a person? (3) A service principal.

Answer. b) A service principal, authenticated with OAuth M2M or workload identity federation Because: r1) A service principal is not tied to a person, and its access can be scoped to exactly what deployment needs.

A service principal gives automation its own identity with only the rights it needs. Databricks recommends OAuth M2M for it, or workload identity federation with no stored secret.

Common wrong answers.

- `a`: A personal token stops working when that person leaves, and it carries all their access.
- `c`: Pull request authors should not need production rights. The deploy runs after merge, as automation.
- `d`: Databricks requires authentication for every API call.

- `b|r2`: Right identity, wrong reason. Personal tokens can technically deploy. The problem is who owns the access.

A10-L3 exercise 2

Hints. (1) OAuth client credentials have two parts. (2) They match the secret names on the right. (3) `CLIENT_ID` and `CLIENT_SECRET`.

Answer.

```yaml
env:
  DATABRICKS_HOST: ${{ vars.DATABRICKS_HOST }}
  DATABRICKS_CLIENT_ID: ${{ secrets.DATABRICKS_CLIENT_ID }}
  DATABRICKS_CLIENT_SECRET: ${{ secrets.DATABRICKS_CLIENT_SECRET }}
```

`DATABRICKS_CLIENT_ID` and `DATABRICKS_CLIENT_SECRET`, with `DATABRICKS_HOST`, let the CLI authenticate as the service principal.

Common wrong answers.

- g1 = `TOKEN`: DATABRICKS_TOKEN is for personal access tokens. OAuth M2M uses the client ID and secret.
- g2 = `PASSWORD`: Service principals authenticate with a client secret, not a password.

A10-L3 exercise 3

Hints. (1) Which setting stops a failing step from failing the job? (2) It sits under the pytest step. (3) Remove `continue-on-error: true`.

Answer. The bug is on line 6.

Fixed version:

```yaml
steps:
  - uses: actions/checkout@v4
  - run: pip install -e . --group dev
  - run: ruff check .
  - run: pytest
```

`continue-on-error: true` makes the step pass whatever pytest returns. Remove it. If tests are flaky, fix or quarantine them in code, and keep CI honest.

Common wrong answers.

- Line 5: Running pytest is right. The line after it changes what a failure means.
- Line 4: The lint step is fine and can fail the job.

A10-L3 exercise 4

Hints. (1) Which checks take seconds? (2) Which starts a JVM? (3) Which talks to the workspace?

Answer.

1. ruff check . and ruff format --check .
2. mypy
3. pytest with a local SparkSession
4. databricks bundle validate -t dev

Lint and types take seconds, tests take a minute or two with Spark, and validation needs the workspace. Failing fast saves CI time and gives quicker feedback.

Common wrong answers.

- Spark tests take the longest. Run the seconds-long checks first, so a typo fails the job quickly.
- Validation needs workspace credentials and a network call. Run the local checks first.

A10-L3 exercise 5

Hints. (1) Checks belong before review and merge. (2) Production changes only after a merge. (3) Credentials never live in files.

Answer.

- Checks on every pull request and on main.
- Deploy to prod only after a merge, needing the checks.
- Protect main with required checks.
- Authenticate as a service principal: federation first, OAuth M2M second, secrets never in files.

Checks gate every pull request, protection makes them binding, deployment follows a merge, and CI authenticates as a service principal without secrets in the repository.

Common wrong answers.

- trigger_deploy = `every_push`: Deploying every branch push puts unreviewed code in production.
- branch_protection = `none`: Without required checks, CI reports problems but cannot stop a merge.
- auth = `pat`: A token in a workflow file is committed to Git, and ties deployment to one person.

- trigger_checks = `nightly`: Nightly checks find problems after they merge. Run them on the pull request.